

Methodenüberschreibung (Method Overriding)

Aspekt	Beschreibung	Java-spezifische Konzepte/Techniken
Definition	Methodenüberschreibung bedeutet, dass eine Methode einer Oberklasse in einer Unterklasse mit derselben Signatur neu definiert wird.	Ermöglicht spezifischeres Verhalten in der Unterklasse. Wird zur Implementierung von Polymorphismus verwendet.
Zweck	Ermöglicht einer Unterklasse, das Verhalten einer geerbten Methode zu ändern oder anzupassen.	Beispiel: <code>Animal</code> hat <code>speak()</code> , aber <code>Dog</code> überschreibt <code>speak()</code> mit einer eigenen Implementierung.
Regeln für Methodenüberschreibung	Die Methode in der Unterklasse muss dieselbe Signatur wie in der Oberklasse haben.	Gleicher Methodenname. Gleiche Parameterliste (Anzahl und Typen). Gleicher oder kompatibler Rückgabotyp
Zugriffsmodifikatoren	Die Sichtbarkeit der überschriebenen Methode darf nicht eingeschränkt werden.	Erlaubt: <code>protected</code> → <code>public</code> (größere Sichtbarkeit). Nicht erlaubt: <code>public</code> → <code>private</code> (weniger Sichtbarkeit).
Rückgabotyp (Covariant Return Type)	Der Rückgabotyp der Methode kann in der Unterklasse eine Unterklasse des ursprünglichen Rückgabetyps sein.	Beispiel: <code>Animal</code> <code>getAnimal()</code> kann in der Unterklasse als <code>Dog</code> <code>getAnimal()</code> überschrieben werden.
Verwendung von <code>@Override</code>	Die Annotation <code>@Override</code> wird empfohlen, um Fehler zu vermeiden.	Erkennt Tippfehler oder falsche Methodensignaturen frühzeitig durch Compiler-Prüfung.
Aufruf der überschriebenen Methode	Die Methode der Oberklasse kann in der Unterklasse weiterhin über <code>super</code> aufgerufen werden.	Beispiel: <code>super.speak();</code> ruft die Originalmethode der Oberklasse auf.
Überschreibung von <code>static</code> -Methoden	<code>static</code> -Methoden können nicht überschrieben, sondern nur versteckt (hidden) werden.	Beispiel: <code>static void show()</code> in der Unterklasse ist keine echte Überschreibung, sondern eine neue Implementierung.
Überschreibung von <code>final</code> -Methoden	Eine Methode, die als <code>final</code> deklariert ist, kann nicht überschrieben werden.	Beispiel: <code>final void display()</code> in der Oberklasse führt zu einem Compilerfehler, wenn <code>display()</code> in der Unterklasse überschrieben wird.
Überschreibung von <code>private</code> -Methoden	<code>private</code> -Methoden sind nicht sichtbar für Unterklassen und können nicht überschrieben werden.	In der Unterklasse ist eine Methode mit demselben Namen keine Überschreibung, sondern eine neue Methode.
Überschreibung von <code>equals()</code> , <code>hashCode()</code> , <code>toString()</code>	Diese Methoden aus <code>Object</code> werden oft überschrieben, um benutzerdefiniertes Verhalten zu definieren.	Beispiel: <code>toString()</code> für menschenlesbare Ausgabe überschreiben.

Aspekt	Beschreibung	Java-spezifische Konzepte/Techniken
Überschreibung von <code>clone()</code>	Muss mit Vorsicht verwendet werden, da <code>clone()</code> in <code>Object</code> <code>protected</code> ist.	Muss explizit <code>public</code> gemacht und <code>Cloneable</code> implementiert werden, sonst <code>CloneNotSupportedException</code> .
Polymorphismus durch Methodenüberschreibung	Eine Unterklasse kann ihre eigene Implementierung einer Methode bereitstellen, wird aber als Oberklasse behandelt.	Beispiel: <code>Animal a = new Dog();</code> <code>a.speak();</code> → <code>Dog</code> 's <code>speak()</code> wird aufgerufen, obwohl <code>a</code> als <code>Animal</code> deklariert ist.
Verwendung in der Java API	Viele Java-Standardbibliotheken nutzen Methodenüberschreibung für Flexibilität.	<code>java.lang.String</code> überschreibt <code>equals()</code> , <code>hashCode()</code> , <code>toString()</code> .

Zusammenfassung

Methodenüberschreibung erlaubt es, das Verhalten einer geerbten Methode in einer Unterklasse neu zu definieren. Dies ist ein zentraler Bestandteil von Polymorphismus in Java. Es gelten dabei strikte Regeln für Methodensignatur, Zugriffsmodifikatoren und Rückgabetypen.